University of Siegen

Department of

Electrical Science - Computer Science

Bachelor's Thesis

Navigational Queries for Forest Straight-Line Programs

Andreas Lizenberger

andreas.lizenberger@student.uni-siegen.de

Supervisor 1: Prof. Dr. Markus Lohrey

Supervisor 2: Dr. Carl Philipp Reh

Handling Period: 26.07.2021 - 26.11.2021

Abstract We implement the navigational queries access, parent, first child, next sibling, size, height, depth, level ancestor, nearest common ancestor and decompress directly for forest straight-line programs, that is a context-free grammar that encodes a tree or forest using algebraic terms that are evaluated in the forest algebra.

Contents

1 Introduction

Every node in a rooted, ordered, Σ-labeled, unranked tree is identifiable by a pre-order number. We can also interpret this pre-order number as a position within the tree. Navigational queries take the current position as input and either retrieve the associated label or subtree, or compute a new position as output. We will discuss the navigational queries access, parent, first-child, next-sibling, height, depth, size, level-ancestor, nearest common ancestor and decompress. These are fundamental algorithms that can be used for navigation in documents that operate on trees, e.g. XML and HTML. There are formalisms that can compress trees exponentially. We can avoid decompression by implementing navigational queries directly on those formalisms.

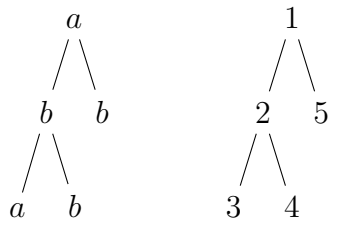

Figure 1: The tree $a(b(ab)b)$ (left) and related pre-order numbers (right). Example queries: access(4) = b, parent(5) = 1, next-sibling(3) = 4.

Forest straight-line programs are context-free grammars that compress rooted, ordered, Σ-labeled, unranked forests, i.e. a (possibly empty) list of trees. The roots of those trees are considered as siblings, even though they do not share a common ancestor. Generally, a context-free grammar $G = (V, \Sigma, S, \rho)$ defines a language $L \subseteq \Sigma^*$ where each word $w \in L$ can be derived by starting with an expression $K = S$ and successively replacing variables in K until $K = w$. V is the set of variables, Σ is an alphabet, $S \in V$ is the starting variable and ρ is a relation that defines what a variable $A \in V$ can be replaced with. An FSLP $F = (V, S, \rho)$ defines a language $L = \{e\}$ with size $|L| = 1$, where e is an algebraic term and ρ is a function that maps variables $A \in V$ to terms with variables $\rho(A)$. We do not derive forests f directly, instead, we construct an algebraic term e that we evaluate in the forest algebra $FA(\Sigma)$, such that the evaluation of e is $[\![e]\!] = f$. The forest algebra [1] constructs forests f by either concatenating forests f_1, f_2 horizontally, denoted by $f = f_1 \boxdot f_2$, or inserting a forest f_1 into a forest context f_2, denoted by $f = f_2 \boxdot\!\!\square f_1$. Forest contexts are forests that have exactly one placeholder x. Furthermore, there is a normal form for FSLPs [2] that restricts the form of the replacement rule for any given variable $A \in V$. We will use the normal form to implement navigational queries for FSLPs.

Navigational queries have been already implemented for top DAGs, an alternative

formalism that compresses rooted, ordered, Σ-labeled, unranked trees. A top DAG $\mathcal{TD}$ is constructed by calculating a top tree $\mathcal{T}$ of a tree t and then applying a specific dag compression algorithm[1] on $\mathcal{T}$ to compute the minimal DAG representation of $\mathcal{T}$. A top tree $\mathcal{T}$ of a tree t is a rooted, ordered, labeled binary tree, where each node of $\mathcal{T}$ corresponds to a specific cluster[2] in t. A cluster C is a subtree in t that is induced by a node v that is the root of C, also called top boundary node of C, leftmost child v_ℓ of v in C, rightmost child v_r of v in C and an optional bottom boundary node u, that is an inner node in C. This means that the cluster C contains v, all siblings between v_ℓ and v_r and all descendants of those siblings, where v_ℓ and v_r and their descendants are included, but C does not contain the descendants of u. We can identify a cluster with the tuple (v, v_ℓ, v_r, u) or (v, v_ℓ, v_r) respectively. There are five types of merges that are allowed when constructing a top DAG. Clusters C_1, C_2 can be merged vertically, denoted by $C_1 \boxplus_k C_2$, $1 \leq k \leq 2$, or horizontally, denoted by $C_1 \boxdot_k C_2$, $3 \leq k \leq 5$. In either case, the clusters must overlap in exactly one node v_x such that v_x is the top boundary node of the cluster C_1 or v_x is the top boundary node of the cluster C_2:

- $C_1 = (v, v_\ell, v_r, v_x),\ C_2 = (v_x, u_\ell, u_r, w),\ C_1 \cup C_2 = (v, v_\ell, v_r, w)$
  $\Rightarrow C_1 \boxplus_1 C_2 := (v, v_\ell, v_r, w)$
- $C_1 = (v, v_\ell, v_r, v_x),\ C_2 = (v_x, u_\ell, u_r),\ C_1 \cup C_2 = (v, v_\ell, v_r)$
  $\Rightarrow C_1 \boxplus_2 C_2 := (v, v_\ell, v_r)$
- $C_1 = (v_x, v_\ell, v_r, u),\ C_2 = (v_x, v_{\ell'}, v_{r'}),\ C_1 \cup C_2 = (v_x, v_\ell, v_{r'}, u)$
  $\Rightarrow C_1 \boxdot_3 C_2 := (v_x, v_\ell, v_{r'}, u)$
- $C_1 = (v_x, v_\ell, v_r),\ C_2 = (v_x, v_{\ell'}, v_{r'}, u),\ C_1 \cup C_2 = (v_x, v_\ell, v_{r'}, u)$
  $\Rightarrow C_1 \boxdot_4 C_2 := (v_x, v_\ell, v_{r'}, u)$
- $C_1 = (v_x, v_\ell, v_r),\ C_2 = (v_x, v_{\ell'}, v_{r'}),\ C_1 \cup C_2 = (v_x, v_\ell, v_{r'})$
  $\Rightarrow C_1 \boxdot_5 C_2 := (v_x, v_\ell, v_{r'})$

Note that in general $C_1 \cup C_2$ is not a cluster. We have to explicitly demand that C_1 and C_2 fit together without creating any gaps. Let $v_1, v_2, ..., v_5$ be children of v and $C_1 = (v, v_1, v_2), C_2 = (v, v_4, v_5)$ be clusters then $C_1 \cup C_2 \neq (v, v_1, v_5)$, because v_3 is missing in $C_1 \cup C_2$. In particular, $C_1 \boxdot_5 C_2$ is not defined.

1.1 Previous Work

Top DAGs have been introduced in [3]. Let t be the tree that is encoded by the top DAG $\mathcal{TD}$. It has been shown that we can compute in linear time w.r.t. $|\mathcal{TD}|$

[1] See: Chapter 2.3 Constructing the top tree in reference [3]
[2] See: Chapter 2.2 Top trees in reference [3]

a data structure with size in $\mathcal{O}(|\mathcal{TD}|)$, such that the navigational queries access, parent, first-child, next-sibling, height, depth, size, level-ancestor and nearest common ancestor have a runtime in $\mathcal{O}(\log |t|)$, and decompressing a subtree t' in t has a runtime in $\mathcal{O}(\log |t| + |t'|)$.
Unfortunately these results cannot be reused for the implementation of navigational queries for FLSPs. Ideally, we would express an FSLP F as an equivalent top DAG $\mathcal{TD}$, but a conversion cannot be done in linear time.
FSLPs have been introduced and compared to top DAGs in [2]. FSLPs operate on a forest algebra with horizontal and vertical concatenations, while top DAGs operate on a cluster algebra using horizontal and vertical merges. It is possible to construct in time $\mathcal{O}(|\mathcal{TD}|)$ an FLSP F for a given top DAG $\mathcal{TD}$ such that F and $\mathcal{TD}$ encode the same tree t and $|F| \in \mathcal{O}(|\mathcal{TD}|)$. On the other hand, the construction of a top DAG $\mathcal{TD}$ for an FSLP F, that encodes a tree t with size $|t| \geq 2$, can be done in time $\mathcal{O}(|\Sigma| \cdot |F|)$ such that $|\mathcal{TD}| \in \mathcal{O}(|\Sigma| \cdot |F|)$, where Σ is the set that contains the labels of the nodes of the tree. There are examples that show that the factor $|\Sigma|$ is unavoidable.
There are two major theorems that allow the implementation of navigational queries for FSLPs. Theorem 1 is a result of the work in [4] and theorem 2 has been shown in [2]. Corollary 3 is a direct consequence of these theorems.

Theorem 1. *For a given FLSP F that encodes a forest f we can construct in linear time w.r.t. $|F|$ an equivalent FSLP F' with size in $\mathcal{O}(|F|)$ and height in $\mathcal{O}(log\ |f|)$.*

Theorem 2. *For a given FSLP F we can construct in linear time w.r.t. $|F|$ an equivalent FSLP F' in normal form with size in $\mathcal{O}(|F|)$. The construction that is used in the proof does not change the height of F' significantly. The height of F' is in $\mathcal{O}(|F|)$.*

Corollary 3. *For a given FSLP F that encodes a forest f we can construct in linear time w.r.t. $|F|$ an equivalent FSLP F' in normal form with size in $\mathcal{O}(|F|)$ and height in $\mathcal{O}(log\ |f|)$.*

1.2 Results of this Bachelor's Thesis

Given an FSLP $F' = (V, S, \rho)$ in normal form as stated in Corollary 3 that encodes a forest f, we will show that we can compute in linear time w.r.t. $|F'|$ a data structure with size in $\mathcal{O}(|F'|)$, such that the navigational queries access, parent, first-child, next-sibling, height, depth, size, level-ancestor and nearest common ancestor have a runtime in $\mathcal{O}(\log |f|)$, and decompressing a subtree t' in f has a runtime in $\mathcal{O}(\log |f| + |t'|)$. We achieve this by iterating over the height of F' and only deriving mandatory variables starting with S.

1.3 Further Research

Forest straight-line programs (FSLP) are a generalization of (string) straight-line programs (SLP). SLPs only use horizontal concatenations on strings and compress strings succinctly. These formalisms are categorized as grammar-based lossless compression schemes that use context-free grammars to compress forests and strings respectively. A grammar can be exponentially smaller than the encoded string, e.g. a grammar with production rules of the form $\rho(A_i) = A_{i-1}A_{i-1}, i > 0$ and $\rho(A_0) = ab$ compresses the string $(ab)^n$, where $n = 2^i$.

The construction of the smallest grammar for a given input is NP-complete and known as the smallest grammar-problem [5]. For an SLP $G = (V, \Sigma, S, \rho)$ the size $|G|$ of G is defined as:

$$|G| = \sum_{A \in V} |\rho(A)|$$

There are effective algorithms that construct relatively small grammars from a given string: Sequitur [6] finds repeats in a string of symbols and introduces variables and rules to replace those repeats, LZW [7] holds a dictionary and slides along the input and adds unseen substrings to the dictionary, Re-Pair [8] finds the most frequently occurring pair of symbols and introduces variables and rules to replace those pairs. An extensive survey of algorithms on compressed strings has been done in [9].

The balancing result for SLPs in [4] has been recently improved upon in [10]. Previously, an SLP G that encodes a string w could be converted in linear time w.r.t. $|G|$ into an equivalent SLP G' with size in $\mathcal{O}(|G|)$ and the height of the unique derivation tree of G' is in $\mathcal{O}(\log |w|)$. Contracting SLPs are introduced in [10] and additionally guarantee that every subtree in the derivation tree of G' has a logarithmic height w.r.t. the number of its leaves. This property has been used in [10] to extend another recent result for FSLPs in [11] that computes for an FLSP F in linear time w.r.t. $|F|$ a data structure with size in $\mathcal{O}(|F|)$, such that the navigation on pointers for variables A can be done in constant runtime: compute the pointer to the root of the first/last tree produced by A, compute the pointer to the first/last child of the current node, compute the pointer to the left/right sibling of the current node, compute the pointer to the parent of the current node, compute the symbol at the current node. Finally, [10] adds that the computation of the pointer to the i-th child of the current node can be done in time $\mathcal{O}(\log d)$ where d is the degree of the current node.

2 Forest Straight-Line Program

2.1 Forests and Trees

Let Σ be an alphabet, with $\Sigma \cap \{\boxdot, \boxplus, \varepsilon, x, (,)\} = \emptyset$, that we fixate for the rest of this bachelor's thesis. It contains all symbols that are permitted to be used as node labels. The symbols in $\{\boxdot, \boxplus, x, (,)\}$ are special characters with regard to forests and terms, and ε denotes the neutral element for concatenation.
From a graph-theoretic standpoint, we consider a forest to be a (possibly empty) sequence of rooted[3], ordered[4], Σ-labeled[5], unranked[6] trees. Let $\mathcal{F}_0(\Sigma)$ be the language over $\Sigma' = \Sigma \uplus \{(,)\}$ that is inductively defined as:

- $\varepsilon \in \mathcal{F}_0(\Sigma)$
- $\forall a \in \Sigma \colon \forall F \in \mathcal{F}_0(\Sigma) \colon a(F) \in \mathcal{F}_0(\Sigma)$
- $\forall F_1, F_2 \in \mathcal{F}_0(\Sigma) \colon F_1 F_2 \in \mathcal{F}_0(\Sigma)$

Let $\mathcal{T}_0(\Sigma) \subseteq \mathcal{F}_0(\Sigma)$ be the language over Σ' that is defined as:

- $\forall a \in \Sigma \colon \forall F \in \mathcal{F}_0(\Sigma) \colon a(F) \in \mathcal{T}_0(\Sigma)$

$\mathcal{F}_0(\Sigma)$ contains all representations of forests as strings and $\mathcal{T}_0(\Sigma)$ contains all representations of trees as strings.
We declare x as a special character that denotes a placeholder for another forest, tree, forest context or tree context. Let $\Sigma'' = \Sigma \uplus \{(,), x\}$. Let $\mathcal{F}_1(\Sigma)$ be the language over Σ'' that is inductively defined as:

- $x \in \mathcal{F}_1(\Sigma)$
- $\forall a \in \Sigma \colon \forall F_x \in \mathcal{F}_1(\Sigma) \colon a(F_x) \in \mathcal{F}_1(\Sigma)$
- $\forall F_1, F_2 \in \mathcal{F}_0(\Sigma) \colon \forall F_x \in \mathcal{F}_1(\Sigma) \colon F_1 F_x F_2 \in \mathcal{F}_1(\Sigma)$

Let $\mathcal{T}_1(\Sigma) \subseteq \mathcal{F}_1(\Sigma)$ be a language over Σ'' that defined as:

- $\forall a \in \Sigma \colon \forall F_x \in \mathcal{F}_1(\Sigma) \colon a(F_x) \in \mathcal{T}_1(\Sigma)$

We call $\mathcal{F}_1(\Sigma)$ the set of all forest contexts, $\mathcal{T}_1(\Sigma)$ the set of all tree contexts and $\mathcal{F}(\Sigma) := \mathcal{F}_0(\Sigma) \uplus \mathcal{F}_1(\Sigma)$ the set of all forests and forest contexts.

[3] Rooted tree: A directed tree (graph) that has a unique vertex r such that a path from r to any other vertex in the graph exists.
[4] Ordered tree: A tree (graph) where the children of any given node are totally ordered.
[5] Σ-labeled tree: A tree (graph) where each vertex has exactly one label in Σ.
[6] Unranked tree: A tree (graph) where the number of children of any given node v is not restricted by the label of v.

2.2 Forest Algebra

An algebra $\mathcal{A}$ over the universe U is a tuple $(U, f_1, f_2, ..., f_n, c_1, c_2, ..., c_m)$, where U is a set, $f_i \colon U^{k_i} \to U$ are k_i-ary, partial functions and $c_j \in U$ are constants, with $1 \leq i \leq n, 1 \leq j \leq m$. Algebras are useful for evaluating terms, i.e. expressions that use the symbols $\underline{f_1}, \underline{f_2}, ..., \underline{f_n}, \underline{c_1}, \underline{c_2}, ..., \underline{c_m}$ and parenthesis $(,)$. Terms are inductively defined as:

- $\underline{c_j}$ is a term
- $e_1, e_2, ..., e_{k_i}$ are terms $\Rightarrow \underline{f_i}(e_1, e_2, ..., e_{k_i})$ is a term

The evaluation of a term e is denoted as $[\![e]\!]$ and inductively defined as:

- $[\![\underline{c_j}]\!] = c_j$
- $e_1, e_2, ..., e_{k_i}$ are terms and $f_i([\![e_1]\!], [\![e_2]\!], ..., [\![e_{k_i}]\!])$ is defined for f_i
  $\Rightarrow [\![\underline{f_i}(e_1, e_2, ..., e_{k_i})]\!] = f_i([\![e_1]\!], [\![e_2]\!], ..., [\![e_{k_i}]\!])$

We will not encounter any terms, where $f_i([\![e_1]\!], [\![e_2]\!], ..., [\![e_{k_i}]\!])$ is not defined for the partial function f_i.
Let $FA(\Sigma) = (\mathcal{F}(\Sigma), f_\boxdot, f_\boxplus, (f_a)_{a\in\Sigma}, f_\varepsilon, f_x)$ denote the forest algebra with Σ-labeled nodes. The functions $f_\boxdot, f_\boxplus$ are 2-ary operations, f_a is a unary operation $\forall a \in \Sigma$, and f_e, f_x are constants. We define for $F, F_1, F_2 \in FA(\Sigma)$:

- $f_\varepsilon = \varepsilon,\ f_x = x$
- $\forall a \in \Sigma \colon f_a(F) = a(F)$
- $F_1 \in \mathcal{F}_0(\Sigma) \vee F_2 \in \mathcal{F}_0(\Sigma) \Rightarrow f_\boxdot(F_1, F_2) := F_1F_2$

For $F, F_x \in \mathcal{F}_1(\Sigma)$ and $F_1, F_2, F_3 \in FA(\Sigma)$ the function $f_\boxplus$ is inductively defined as:

- $F = x \Rightarrow f_\boxplus(F, F_1) = F_1$
- $\forall a \in \Sigma \colon F = a(F_x) \Rightarrow f_\boxplus(F, F_2) = a(f_\boxplus(F_x, F_2))$
- $F = F_1F_xF_2 \Rightarrow f_\boxplus(F, F_3) = F_1f_\boxplus(F_x, F_3)F_2$

Let $\Sigma''' = \Sigma \uplus \{\boxdot, \boxplus, \varepsilon, x, (,)\}$ be the set of symbols that we construct terms with such that $\underline{f_\boxdot} = \boxdot,\ \underline{f_\boxplus} = \boxplus,\ \underline{f_\varepsilon} = \varepsilon,\ \underline{f_x} = x,\ \underline{f_a} = a,\ \forall a \in \Sigma$. We write $e_1 \boxdot e_2$ instead of $\boxdot(e_1, e_2)$ and $e_1 \boxplus e_2$ instead of $\boxplus(e_1, e_2)$.

2.3 Context-free Grammars over Algebras

Let V be a set of variables and $\mathcal{A} = (U, f_1, f_2, ..., f_n, c_1, c_2, ..., c_m)$ be an algebra. Terms with variables are inductively defined as:

- every term is a term with variables
- $\forall A \in V$: A is a term with variables
- $e_1, e_2, ..., e_{k_i}$ are terms with variables $\Rightarrow \underline{f_i}(e_1, e_2, ..., e_{k_i})$ is a term with variables

Let ρ be a relation that contains tuples of the form (A, B), where $A \in V$ is a variable and B is a term with variables. The relation $A\rho B$ expresses that it is legal to replace any occurrence of A with B in a term with variables K. A context-free grammar G over an algebra $\mathcal{A}$ is a tuple (V, S, ρ), where $S \in V$ is the starting variable. Depending on ρ, it is possible to derive terms from G by repeatedly replacing all variables A in S and in all subsequent terms with variables K with a right-hand side B.
If ρ is right-unique and left-total we write $\rho(A) = B$ instead of $A\rho B$ and treat ρ like a function in all regards. Additionally, if $(V, \{(A, B) \in V^2 \mid B \text{ occurs in } \rho(A)\})$ is a directed acyclic graph, we can define the evaluation of terms with variables. If e is a term with variables then $[\![e]\!]_G$ is the evaluation of e in G and defined as:

- e is a term $\Rightarrow [\![e]\!]_G = [\![e]\!]$
- $\forall A \in V$: $[\![A]\!]_G = [\![\rho(A)]\!]_G$
- $e_1, e_2, ..., e_{k_i}$ are terms with variables and $f_i([\![e_1]\!]_G, [\![e_2]\!]_G, ..., [\![e_{k_i}]\!]_G)$ is defined for $f_i \Rightarrow [\![\underline{f_i}(e_1, e_2, ..., e_{k_i})]\!]_G = f_i([\![e_1]\!]_G, [\![e_2]\!]_G, ..., [\![e_{k_i}]\!]_G)$

2.4 Forest Straight-Line Program (FSLP)

An FSLP $F = (V, S, \rho)$ is a context-free grammar over the forest algebra $FA(\Sigma)$ such that:

- ρ is right-unique and left-total
- $(V, \{(A, B) \in V^2 \mid B \text{ occurs in } \rho(A)\})$ is a directed acyclic graph
- $[\![S]\!]_F$ is defined and $[\![S]\!]_F \in \mathcal{F}_0(\Sigma)$

We consider an FSLP $F = (V, S, \rho)$ to be in normal form, if $\forall A \in V$ it holds true that $\rho(A)$ is of one of the following mutually exclusive forms:

- $\rho(A) = B \boxplus C, \quad B, C \in V, \text{ with } [\![B]\!]_F \in \mathcal{T}_1(\Sigma), [\![C]\!]_F \in \mathcal{T}_0(\Sigma)$
- $\rho(A) = B \boxplus C, \quad B, C \in V, \text{ with } [\![B]\!]_F, [\![C]\!]_F \in \mathcal{T}_1(\Sigma)$

- $\rho(A) = a(B \boxdot x \boxdot C), \quad a \in \Sigma$ and $B, C \in V$, with $[\![B]\!]_F, [\![C]\!]_F \in \mathcal{F}_0(\Sigma)$
- $\rho(A) = B \boxdot C, \quad a \in \Sigma$ and $B, C \in V$, with $[\![B]\!]_F, [\![C]\!]_F \in \mathcal{F}_0(\Sigma)$
- $\rho(A) = a(B), \quad a \in \Sigma$ and $B \in V$, with $[\![B]\!]_F \in \mathcal{F}_0(\Sigma)$
- $\rho(A) = \varepsilon$

The spine of a variable A, with $[\![A]\!]_F \in \mathcal{T}_1(\Sigma)$ and $\rho(A) = B \boxplus C$, is the path from the root of $[\![A]\!]_F$ to x.

Example

Let $F = (V, S, \rho)$ be an FSLP, where $V = \{S, A, A_1, A_2, F, B, C, E, D, G, H, Z\}$ and ρ is a function such that:

- $\rho(S) = A \boxdot G$
- $\rho(A) = A_1 \boxplus D$
- $\rho(A_1) = A_2 \boxplus B$
- $\rho(A_2) = a(Z \boxdot x \boxdot F)$
- $\rho(F) = f(Z)$
- $\rho(B) = b(C \boxdot x \boxdot E)$
- $\rho(C) = c(Z)$
- $\rho(E) = e(Z)$
- $\rho(D) = d(Z)$
- $\rho(G) = g(H)$
- $\rho(H) = h(Z)$
- $\rho(Z) = \varepsilon$

F is in normal form and encodes the forest $[\![S]\!]_F = a(\varepsilon b(c(\varepsilon)d(\varepsilon)e(\varepsilon))f(\varepsilon))g(h(\varepsilon))$.

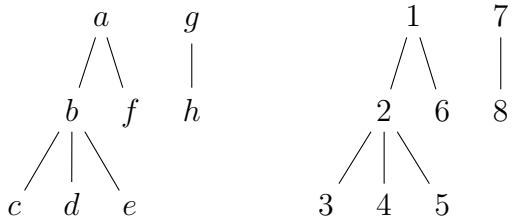

Figure 2: The forest $[\![S]\!]_F$ (left) and related pre-order numbers (right).

If we omit ε we get a more readable presentation: $[\![S]\!]_F = a(b(cde)f)g(h)$.

3 Preprocessing an FSLP

3.1 FSLP in Normal Form

It suffices to define the navigational queries for FSLPs in normal form that have a height that is logarithmic in the size of the encoded forest. We can convert any FSLP to fit these criteria as Corollary 3 shows. The algorithms that we present in the next chapter iterate over the height of the FSLP and perform computations using the data structure $(F, t, h_s, h, n, n_\ell)$ that will be defined in this chapter.
For the rest of this bachelor's thesis, we can assume that $F = (V, S, \rho)$ denotes an FSLP in normal form such that the height of F is in $\mathcal{O}(\log N)$, where N is the size of $[\![S]\!]_F$, and $A, B, C \in V$ are variables and $a \in \Sigma$ is a label.

3.2 Forest Type

For a variable $A \in V$ the forest type t means that $[\![A]\!]_F \in \mathcal{F}_{t(A)}(\Sigma)$. If A evaluates to a forest then $t(A) = 0$, otherwise A evaluates to a forest context and $t(A) = 1$. We need to count the number of occurrences of the placeholder x in $\rho(A)$.
Let $t\colon V \to \{0, 1\}$ be a function that is inductively defined as:

- $\rho(A) = B \boxplus C \Rightarrow t(A) = t(C)$
- $\rho(A) = a(B \boxdot x \boxdot C) \Rightarrow t(A) = 1$
- $\rho(A) = B \boxdot C \Rightarrow t(A) = 0$
- $\rho(A) = a(B) \Rightarrow t(A) = 0$
- $\rho(A) = \varepsilon \Rightarrow t(A) = 0$

Note that by definition of the normal form the evaluations $[\![B \boxdot C]\!]_F$ and $[\![a(B)]\!]_F$ must be in $\mathcal{F}_0(\Sigma)$.

3.3 Forest Height

For a variable $A \in V$ the forest height h means that the height of $[\![A]\!]_F$ is $h(A)$. The height of a forest is the length of the longest path from a root to a leaf. The empty forest ε has the height -1.
For a variable $A \in V$, with $[\![A]\!]_F \in \mathcal{F}_1(\Sigma)$, the spine height h_s means that the length of the path from the root of $[\![A]\!]_F$ to x is $h_s(A)$.
Let $h_s\colon V \to \mathbb{N}_0 \cup \{-1\}$ be a partial function that is inductively defined as:

- $\rho(A) = a(B \boxdot x \boxdot C) \Rightarrow h_s(A) = 1$
- $t(A) = 1 \wedge \rho(A) = B \boxplus C \Rightarrow h_s(A) = h_s(B) + h_s(C)$

Let $h\colon V \to \mathbb{N}_0 \cup \{-1\}$ be a function that is inductively defined as:

- $\rho(A) = B \boxplus C \Rightarrow h(A) = max\{h(B), h_s(B) + h(C)\}$
- $\rho(A) = a(B \boxdot x \boxdot C) \Rightarrow h(A) = 1 + max\{h(B), h(C)\}$
- $\rho(A) = B \boxdot C \Rightarrow h(A) = max\{h(B), h(C)\}$
- $\rho(A) = a(B) \Rightarrow h(A) = 1 + h(B)$
- $\rho(A) = \varepsilon \Rightarrow h(A) = -1$

3.4 Forest Size

For a variable $A \in V$ the forest size n means that $|[\![A]\!]_F| = n(A)$.
Let $n\colon V \to \mathbb{N}_0$ be a function that is inductively defined as:

- $\rho(A) = B \boxplus C \Rightarrow n(A) = n(B) + n(C)$
- $\rho(A) = a(B \boxdot x \boxdot C) \Rightarrow n(A) = 1 + n(B) + n(C)$
- $\rho(A) = B \boxdot C \Rightarrow n(A) = n(B) + n(C)$
- $\rho(A) = a(B) \Rightarrow n(A) = 1 + n(B)$
- $\rho(A) = \varepsilon \Rightarrow n(A) = 0$

For a variable $A \in V$, with $[\![A]\!]_F \in \mathcal{F}_1(\Sigma)$, the size n_ℓ of the forest to the left of the spine means that $n_\ell(A) + 1$ is the pre-order number of x in $[\![A]\!]_F$.
Let $n_\ell\colon V \to \mathbb{N}_0$ be a partial function that is inductively defined as:

- $\rho(A) = a(B \boxdot x \boxdot C) \Rightarrow n_\ell(A) = n(A) - n(C)$
- $t(A) = 1 \wedge \rho(A) = B \boxplus C \Rightarrow n_\ell(A) = n_\ell(B) + n_\ell(C)$

3.5 Data Structure

The partial functions t, h_s, h, n, n_ℓ can be computed in linear time w.r.t. $|F|$ by traversing F in post-order. The resulting data structure $(F, t, h_s, h, n, n_\ell)$ has a size in $\mathcal{O}(|F|)$.

Example

Let $F = (V, S, \rho)$ be the FSLP that we used in the example from chapter 2.4.

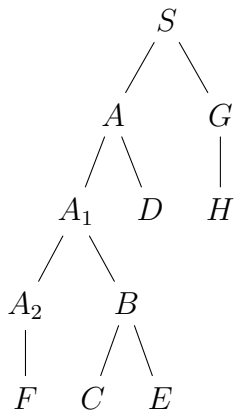

Figure 3: The dag $(V, \{(V_1, V_2) \in V^2 \mid V_2 \text{ occurs in } \rho(V_1)\})$. The variables A_2, F, C, E, D, H are connected with the variable Z that is omitted.

We traverse the dag stated above in post-order and compute:

$$\{V_0 \in V \mid t(V_0) = 0\} = \{Z, F, C, E, D, A, H, G, S\}$$
$$\{V_1 \in V \mid t(V_1) = 1\} = \{A_2, B, A_1\}$$

$$h_s(A_2) = 1$$
$$h_s(B) = 1$$
$$h_s(A_1) = h_s(A_2) + h_s(B) = 2$$

$$h(Z) = -1$$
$$h(F) = 1 + h(Z) = 0$$
$$h(A_2) = 1 + max\{h(Z), h(F)\} = 1$$
$$h(C) = 1 + h(Z) = 0$$
$$h(E) = 1 + h(Z) = 0$$
$$h(B) = 1 + max\{h(C), h(E)\} = 1$$
$$h(A_1) = max\{h(A_2), h_s(A_2) + h(B)\} = 2$$
$$h(D) = 1 + h(Z) = 0$$
$$h(A) = max\{h(A_1), h_s(A_1) + h(D)\} = 2$$
$$h(H) = 1 + h(Z) = 0$$
$$h(G) = 1 + h(H) = 1$$
$$h(S) = max\{h(A), h(G)\} = 2$$

$$n(Z) = 0$$
$$n(F) = 1 + n(Z) = 1$$
$$n(A_2) = 1 + n(Z) + n(F) = 2$$

$$n(C) = 1 + n(Z) = 1$$
$$n(E) = 1 + n(Z) = 1$$
$$n(B) = 1 + n(C) + n(E) = 3$$
$$n(A_1) = n(A_2) + n(B) = 5$$
$$n(D) = 1 + n(Z) = 1$$
$$n(A) = n(A_1) + n(D) = 6$$
$$n(H) = 1 + n(Z) = 1$$
$$n(G) = 1 + n(H) = 2$$
$$n(S) = n(A) + n(G) = 8$$

$$n_\ell(A_2) = n(A_2) - n(F) = 1$$
$$n_\ell(B) = n(B) - n(E) = 2$$
$$n_\ell(A_1) = n_\ell(A_2) + n_\ell(B) = 3$$

4 Navigational Queries

The upcoming algorithms solve their respective problem by evaluating the starting variable S until the node with pre-order number $k_0 \geq 1$ is reached. Terms that do not contribute to finding k_0 are skipped and do not need to be evalutated or stored. The tuple (k, K, n_x) suffices to navigate through the underlying forest $[\![S]\!]_F$, where k is the local pre-order number of the node with pre-order number k_0 within the current subgraph $[\![K]\!]_F$ and n_x is the number of nodes that would replace x when $\rho(K) = a(B \boxdot x \boxdot C)$. Every algorithm stores additional data that is updated in each iteration. Once k_0 is found, most algorithms stop and store the result in the variable m. In any case, the algorithms perform z iterations at worst, where z is the height of F. The algorithms have therefore a runtime in $\mathcal{O}(\log |[\![S]\!]_F|)$.

4.1 Access

The algorithm calculates the label m of the node with pre-order number $k_0 \in \mathbb{N}_1$ in the forest $[\![S]\!]_F$. If k_0 is not a pre-order number in the forest $[\![S]\!]_F$ then the algorithm terminates and sets $m = \varepsilon$.

Programming Variables

- k stores the local pre-order number
- K stores the current variable that needs to be evaluated
- n_x stores the number of nodes that would replace x in $\rho(K) = a(B \boxdot x \boxdot C)$
- m stores the label associated with k_0

Algorithm

set $k = k_0$, $K = S$, $n_x = 0$, $m = \varepsilon$
if $k > n(K)$ then
  STOP
endif
begin loop:
  begin case1: $\rho(K) = B \boxplus C$
    if $k \leq n_\ell(B)$ or $k > n_\ell(B) + n(C) + n_x$ then
      set $K = B$, $n_x = n_x + n(C)$
    else
      set $K = C$, $k = k - n_\ell(B)$
    endif
  end case1

```
  begin case2: ρ(K) = a(B ⊡ x ⊡ C)
    if k ≤ 1 then
      set m = a
      STOP
    elseif k ≤ 1 + n(B) then
      set K = B, k = k − 1
    else
      set K = C, k = k − 1 − n(B) − n_x
    endif
    set n_x = 0
  end case2
  begin case3: ρ(K) = B ⊡ C
    if k ≤ n(B) then
      set K = B
    else
      set K = C, k = k − n(B)
    endif
  end case3
  begin case4: ρ(K) = a(B)
    if k ≤ 1 then
      set m = a
      STOP
    else
      set K = B, k = k − 1
    endif
  end case4
end loop
```

Example

Let $F = (V, S, \rho)$ be the FSLP that we used in the example from chapter 2.4. We compute the label m for pre-order number $k_0 = 5$:

Initialization: $k = 5,\ K = S,\ n_x = 0,\ m = \varepsilon$

Iteration 1: case3: $\rho(K) = A \boxdot G$
$k = 5 \leq 6 = n(A)$
set $K = A$
Context: $k = 5,\ K = A,\ n_x = 0,\ m = \varepsilon$

Iteration 2: case1: $\rho(K) = A_1 \boxplus D$
$k = 5 > 4 = 3 + 1 + 0 = n_\ell(A_1) + n(D) + n_x$
set $K = A_1,\ n_x = n_x + n(D) = 0 + 1 = 1$
Context: $k = 5,\ K = A_1,\ n_x = 1,\ m = \varepsilon$

Iteration 3: case1: $\rho(K) = A_2 \boxplus B$
$k = 5 > 1 = n_\ell(A_2)$ and $k = 5 \leq 5 = 1 + 3 + 1 = n_\ell(A_2) + n(B) + n_x$
set $K = B,\ k = k - n_\ell(A_2) = 5 - 1 = 4$
Context: $k = 4,\ K = B,\ n_x = 1,\ m = \varepsilon$

Iteration 4: case2: $\rho(K) = b(C \boxdot x \boxdot E)$
$k = 4 > 2 = 1 + n(C)$
set $K = E,\ k = k - 1 - n(C) - n_x = 4 - 1 - 1 - 1 = 1,\ n_x = 0$
Context: $k = 1,\ K = E,\ n_x = 0,\ m = \varepsilon$

Iteration 5: case4: $\rho(K) = e(Z)$
$k = 1 \leq 1$
set $m = e$
STOP

Design

The algorithm has a simple structure. It selects the current case and then checks which variable needs to be evaluated next. For example, if $\rho(K) = B \boxplus C$ then case1 is selected. The node in $[\![S]\!]_F$ with pre-order number k_0 has the same label as the node in $[\![K]\!]_F$ with pre-order number k. If $k \leq n_\ell(B)$ or $k > n_\ell(B) + n(C) + n_x$ then we need to evaluate B next. We need to set $K = B$ to start the next iteration. It is not enough to check for $k \leq n_\ell(B)$ as the next example shows. Let us assume that $\rho(B) = a(B' \boxdot x \boxdot C')$. The function n_ℓ tells us how many nodes are produced left of x, that is $n(B')$, and by the spine of B, that is 1 namely a, so in total $n_\ell(B) = 1 + n(B')$. We still need to take into account that the node with the local pre-order number k_0 might be produced by C'. This explains why we need to check for $k > n_\ell(B) + n(C) + n_x$.
From now on, let us underline the term with variables that needs to be evaluated next. This saves us from writing down the associated if statements.
Let us assume that $K = B$ and $\rho(B) = a(B' \boxdot x \boxdot \underline{C'})$. We need to access the number n_x of nodes that would replace x. Unfortunately, we need to have stored this number in the previous iteration. It would be inaccessible otherwise, since the algorithm only stores the current variable. This explains why we set $n_x = n_x + n(C)$ in case1.

4.2 Parent

The algorithm calculates the pre-order number m of the parent of the node with pre-order number $k_0 \in \mathbb{N}_1$ in the forest $[\![S]\!]_F$. If k_0 is not a pre-order number in the forest $[\![S]\!]_F$ or k_0 is a root then the algorithm terminates and sets $m = \varepsilon$.

Programming Variables

- k stores the local pre-order number
- K stores the current variable that needs to be evaluated
- n_x stores the number of nodes that would replace x in $\rho(K) = a(B \boxdot x \boxdot C)$
- m stores the pre-order number of the parent associated with k_0
- M stores the variable B that contains the parent when $\rho(K) = B \boxplus C$
- m_1 stores the global pre-order number of the first node that is produced by M

Algorithm

set $k = k_0,\ K = S,\ n_x = 0,\ m = \varepsilon,\ M = \varepsilon,\ m_1 = \varepsilon$
if $k > n(K)$ then
  STOP
endif
begin loop:
  begin case1: $\rho(K) = B \boxplus C$
    if $k \leq n_\ell(B)$ or $k > n_\ell(B) + n(C) + n_x$ then
      set $K = B,\ n_x = n_x + n(C)$
    else
      set $M = B,\ m_1 = k_0 - k + 1,\ K = C,\ k = k - n_\ell(B)$
    endif
  end case1
  begin case2: $\rho(K) = a(B \boxdot x \boxdot C)$
    if $k \leq 1$ then
      SEARCH
    elseif $k \leq 1 + n(B)$ then
      set $M = \varepsilon,\ m = k_0 - k + 1,\ K = B,\ k = k - 1$
    else
      set $M = \varepsilon,\ m = k_0 - k + 1,\ K = C,\ k = k - 1 - n(B) - n_x$
    endif
    set $n_x = 0$

```
  end case2
  begin case3: ρ(K) = B ⊡ C
    if k ≤ n(B) then
      set K = B
    else
      set K = C, k = k − n(B)
    endif
  end case3
  begin case4: ρ(K) = a(B)
    if k ≤ 1 then
      SEARCH
    else
      set M = ε, m = k_0 − k + 1, K = B, k = k − 1
    endif
  end case4
end loop
beginfunction SEARCH:
  if M ≠ ε then
    begin loop:
      begin case1: ρ(M) = B ◫ C
        set M = C, m_1 = m_1 + n_ℓ(B)
      end case1
      begin case2: ρ(M) = a(B ⊡ x ⊡ C)
        set m = m_1
        STOP
      end case2
    end loop
  endif
  STOP
endfunction
```

Example

Let $F = (V, S, \rho)$ be the FSLP that we used in the example from chapter 2.4. We compute the pre-order number m of the parent of the node with pre-order number $k_0 = 4$:

Initialization: $k = 4,\ K = S,\ n_x = 0,\ m = \varepsilon,\ M = \varepsilon,\ m_1 = \varepsilon$

Iteration 1: case3: $\rho(K) = A \boxdot G$

$k = 4 \leq 6 = n(A)$
set $K = A$
Context: $k = 4,\ K = A,\ n_x = 0,\ m = \varepsilon,\ M = \varepsilon,\ m_1 = \varepsilon$

Iteration 2: case1: $\rho(K) = A_1 \boxplus D$
$k = 4 > 3 = n_\ell(A_1)$ and $k = 4 \leq 4 = 3 + 1 + 0 = n_\ell(A_1) + n(D) + n_x$
set $M = A_1,\ m_1 = k_0 - k + 1 = 4 - 4 + 1 = 1,\ K = D,\ k = k - n_\ell(A_1) = 4 - 3 = 1$
Context: $k = 1,\ K = D,\ n_x = 0,\ m = \varepsilon,\ M = A_1,\ m_1 = 1$

Iteration 3: case4: $\rho(K) = d(Z)$
$k = 1 \leq 1$
SEARCH
Context: $k = 1,\ K = D,\ n_x = 0,\ m = \varepsilon,\ M = A_1,\ m_1 = 1$

Iteration 4: case1: $\rho(M) = A_2 \boxplus B$
set $M = B,\ m_1 = m_1 + n_\ell(A_2) = 1 + 1 = 2$
Context: $k = 1,\ K = D,\ n_x = 0,\ m = \varepsilon,\ M = B,\ m_1 = 2$

Iteration 5: case2: $\rho(M) = b(C \boxdot x \boxdot E)$
set $m = m_1 = 2$
STOP

Design

This algorithm has a similar structure to the algorithm that computes access. As we navigate through the FSLP to find the node with the pre-order number k_0, we have to keep track of potential parent nodes and store their pre-order number in m. Unfortunately, there are cases where we skip nodes entirely and cannot directly keep track of the parent node. In that case we need to store the variable that would produce the parent in M.
Let us underline the term with variables that needs to be evaluated to find the node with pre-order number k_0. For example, $\rho(K) = B \boxplus \underline{C}$ expresses that the node is produced by C. If $\rho(B) = a(B' \boxdot x \boxdot C')$ and $\rho(C) = \underline{b}(Z)$ then a is the parent of b. We need to set $K = C$ and $M = B$. Additionally, we need to set $m_1 = k_0 - k + 1$. This means that m_1 stores the pre-order number of the first node that is produced by M. We will need that information later on.
Let us discuss the next iteration where $\rho(K) = \underline{b}(Z)$. The algorithm has stored the parent in M. Since $M = B$ and B produces a tree context, we know that the parent we are looking for is the parent of x in $[\![B]\!]_F$. The subroutine SEARCH navigates to that parent and computes the pre-order number using m_1. Here, we just need to

set $m = m_1$ and call STOP because m_1 already points at the first label created by B which is a.
The algorithm is constructed in a way such that exactly one of the following cases is true:

- $M = \varepsilon = m \Rightarrow$ there is no parent
- $M = \varepsilon \neq m \Rightarrow$ the pre-order number of the parent is m
- $M \neq \varepsilon \Rightarrow$ the parent is constructed by M

4.3 First Child

The algorithm calculates the pre-order number m of the first child of the node with pre-order number $k_0 \in \mathbb{N}_1$ in the forest $[\![S]\!]_F$. If k_0 is not a pre-order number in the forest $[\![S]\!]_F$ or k_0 is a leaf then the algorithm terminates and sets $m = \varepsilon$.

Programming Variables

- k stores the local pre-order number
- K stores the current variable that needs to be evaluated
- n_x stores the number of nodes that would replace x in $\rho(K) = a(B \boxdot x \boxdot C)$
- m stores the pre-order number of the first child associated with k_0

Algorithm

set $k = k_0,\ K = S,\ n_x = 0,\ m = \varepsilon$
if $k > n(K)$ then
  STOP
endif
begin loop:
  begin case1: $\rho(K) = B \boxplus C$
    if $k \leq n_\ell(B)$ or $k > n_\ell(B) + n(C) + n_x$ then
      set $K = B,\ n_x = n_x + n(C)$
    else
      set $K = C,\ k = k - n_\ell(B)$
    endif
  end case1
  begin case2: $\rho(K) = a(B \boxdot x \boxdot C)$
    if $k \leq 1$ then
      if $0 < n(B) + n_x + n(C)$ then

```
        set  m = k_0 + 1
      endif
      STOP
    elseif  k ≤ 1 + n(B)  then
      set  K = B, k = k − 1
    else
      set  K = C, k = k − 1 − n(B) − n_x
    endif
    set  n_x = 0
  end  case2
  begin  case3: ρ(K) = B ⊡ C
    if  k ≤ n(B)  then
      set  K = B
    else
      set  K = C, k = k − n(B)
    endif
  end  case3
  begin  case4: ρ(K) = a(B)
    if  k ≤ 1  then
      if  n(B) ≠ 0  then
        set  m = k_0 + 1
      endif
      STOP
    else
      set  K = B, k = k − 1
    endif
  end  case4
end  loop
```

Design

This algorithm behaves like the algorithm for access and finds the node with pre-order number k_0. The pre-order number of the first child, if it exists, is $k_0 + 1$. For example, if $\rho(K) = \underline{a}(B)$ and $n(B) \neq 0$ then B produces at least one node. We can set $m = k_0 + 1$ and call STOP.

4.4 Next Sibling

The algorithm calculates the pre-order number m of the sibling to the right of the node with pre-order number $k_0 \in \mathbb{N}_1$ in the forest $[\![S]\!]_F$. If k_0 is not a pre-

order number in the forest $[\![S]\!]_F$ or k_0 does not have a sibling to the right then the algorithm terminates and sets $m = \varepsilon$.

Programming Variables

- k stores the local pre-order number
- K stores the current variable that needs to be evaluated
- n_x stores the number of nodes that would replace x in $\rho(K) = a(B \boxdot x \boxdot C)$
- m stores the pre-order number of the next sibling associated with k_0
- M stores the variable B that contains the next sibling when $\rho(K) = B \boxplus C$
- m_1 stores the global pre-order number of the first node that is produced by M
- m_2 stores the number of nodes that would replace x in $\rho(M) = a(B \boxdot x \boxdot C)$ during function SEARCH

Algorithm

```
set k = k_0, K = S, n_x = 0, m = ε, M = ε, m_1 = ε, m_2 = ε
if k > n(K) then
  STOP
endif
begin loop:
  begin case1: ρ(K) = B ⊞ C
    if k ≤ n_ℓ(B) or k > n_ℓ(B) + n(C) + n_x then
      set K = B, n_x = n_x + n(C)
    else
      set M = B, m_1 = k_0 − k + 1, m = ε
      set K = C, k = k − n_ℓ(B)
    endif
  end case1
  begin case2: ρ(K) = a(B ⊡ x ⊡ C)
    if k ≤ 1 then
      set m_2 = 1 + n(B) + n(C) + n_x
      SEARCH
    elseif k ≤ 1 + n(B) then
      if n_x ≠ 0 or n(C) ≠ 0 then
        M = ε, m = k_0 − k + 2 + n(B)
      else
```

```
        M = ε, m = ε
      endif
      set K = B, k = k − 1
    else
      set M = ε, m = ε, K = C, k = k − 1 − n(B) − n_x
    endif
    set n_x = 0
  end case2
  begin case3: ρ(K) = B ⊡ C
    if k ≤ n(B) then
      if n(C) ≠ 0 then
        set M = ε, m = k_0 − k + n(B) + 1
      endif
      set K = B
    else
      set K = C, k = k − n(B)
    endif
  end case3
  begin case4: ρ(K) = a(B)
    if k ≤ 1 then
      set m_2 = 1 + n(B)
      SEARCH
    else
      set M = ε, m = ε, K = B, k = k − 1
    endif
  end case4
end loop
beginfunction SEARCH:
  if M ≠ ε then
    begin loop:
      begin case1: ρ(M) = B ◫ C
        set M = C, m_1 = m_1 + n_ℓ(B)
      end case1
      begin case2: ρ(M) = a(B ⊡ x ⊡ C)
        if n(C) ≠ 0 then
          set m = m_1 + 1 + n(B) + m_2
        endif
        STOP
      end case2
```

```
    end loop
  endif
  STOP
endfunction
```

Example

Let $F = (V, S, \rho)$ be the FSLP that we used in the example from chapter 2.4. We compute the pre-order number m of the next sibling of the node with pre-order number $k_0 = 4$:

Initialization: $k = 4,\ K = S,\ n_x = 0,\ m = \varepsilon,\ M = \varepsilon,\ m_1 = \varepsilon,\ m_2 = \varepsilon$

Iteration 1: case3: $\rho(K) = A \boxdot G$
$k = 4 \leq 6 = n(A)$ and $n(G) = 2 \neq 0$
set $M = \varepsilon,\ m = k_0 - k + n(A) + 1 = 4 - 4 + 6 + 1 = 7,\ K = A$
Context: $k = 4,\ K = A,\ n_x = 0,\ m = 7,\ M = \varepsilon,\ m_1 = \varepsilon,\ m_2 = \varepsilon$

Iteration 2: case1: $\rho(K) = A_1 \boxplus D$
$k = 4 > 3 = n_\ell(A_1)$ and $k = 4 \leq 4 = 3 + 1 + 0 = n_\ell(A_1) + n(D) + n_x$
set $M = A_1,\ m_1 = k_0 - k + 1 = 4 - 4 + 1 = 1,\ m = \varepsilon$
set $K = D,\ k = k - n_\ell(A_1) = 4 - 3 = 1$
Context: $k = 1,\ K = D,\ n_x = 0,\ m = \varepsilon,\ M = A_1,\ m_1 = 1,\ m_2 = \varepsilon$

Iteration 3: case4: $\rho(K) = d(Z)$
$k = 1 \leq 1$
set $m_2 = 1 + n(Z) = 1 + 0 = 1$
SEARCH
Context: $k = 1,\ K = D,\ n_x = 0,\ m = \varepsilon,\ M = A_1,\ m_1 = 1,\ m_2 = 1$

Iteration 4: case1: $\rho(M) = A_2 \boxplus B$
set $M = B,\ m_1 = m_1 + n_\ell(A_2) = 1 + 1 = 2$
Context: $k = 1,\ K = D,\ n_x = 0,\ m = \varepsilon,\ M = B,\ m_1 = 2,\ m_2 = 1$

Iteration 5: case2: $\rho(M) = b(C \boxdot x \boxdot E)$
$n(E) = 1 \neq 0$
set $m = m_1 + 1 + n(C) + m_2 = 2 + 1 + 1 + 1 = 5$
STOP

Design

This algorithm has a similar structure to the algorithm that computes parent. We need to navigate through the FSLP and keep track of potential sibling nodes and store their pre-order number in m or store the variable that produces the next sibling in M. If $\rho(K) = B \boxplus \underline{C}$ and $\rho(C) = \underline{b}(C' \boxdot x \boxdot B')$ then the next sibling of b ist produced by $M = B$. We will need to set $m_1 = k_0 - k + 1$. This means that m_1 stores the pre-order number of the first node that is produced by M. We will need this information when we run the subroutine SEARCH.
We continue and navigate to $K = C$. The node with pre-order number k_0 is b. We will need to set $m_2 = 1 + n(B) + n(C) + n_x$. This means that in the tree context $[\![B]\!]_F$ there are m_2 many nodes that would replace x. We can now use the tuple (m, M, m_1, m_2) to find a potential next sibling in $[\![B]\!]_F$.
SEARCH navigates through the spine of the tree context until we arrive at the variable that produces x in $[\![B]\!]_F$. Let us assume that $\rho(M) = a(B'' \boxdot x \boxdot C'')$. The next sibling of b must be a node that is produced by C''. Currently, m_1 stores the pre-order number of a and m_2 stores the number of nodes that would replace x. The pre-order number of the next sibling is $m = m_1 + 1 + n(B) + m_2$. If $n(C'') = 0$ then C'' does not produce any nodes and there is no next sibling.
The algorithm is constructed in a way such that exactly one of the following cases is true:

- $M = \varepsilon = m \Rightarrow$ there is no next sibling
- $M = \varepsilon \neq m \Rightarrow$ the pre-order number of the next sibling is m
- $M \neq \varepsilon = m \Rightarrow$ the next sibling is constructed by M

Let us analyze another important example. Let $\rho(K) = a(\underline{B} \boxdot x \boxdot C)$. If $n_x \neq 0$ then the next sibling could be produced by a variable that would replace x. The specific variable does not matter. We are only interested in the pre-order number that is $m = k_0 - k + 2 + n(B)$. Remember, $k_0 - k + 1$ is the pre-order number of a in $[\![S]\!]_F$. The pre-order number of the first node that would replace x is therefore m. If $n_x = 0$ and $n(C) \neq 0$ then the same argument can be made for C instead. If $n_x + n(C) = 0$ then a only has one child node and there can be no next sibling. We reset $M = \varepsilon = m$.

4.5 Size

The algorithm calculates the number of nodes m in the tree that is induced by the node with pre-order number $k_0 \in \mathbb{N}_1$ in the forest $[\![S]\!]_F$. If k_0 is not a pre-order number in the forest $[\![S]\!]_F$ then the algorithm terminates and sets $m = \varepsilon$.

Programming Variables

- k stores the local pre-order number
- K stores the current variable that needs to be evaluated
- n_x stores the number of nodes that would replace x in $\rho(K) = a(B \boxdot x \boxdot C)$
- m stores the size of the tree whose root is associated with k_0

Algorithm

```
set k = k_0, K = S, n_x = 0, m = ε
if k > n(K) then
  STOP
endif
begin loop:
  begin case1: ρ(K) = B ◫ C
    if k ≤ n_ℓ(B) or k > n_ℓ(B) + n(C) + n_x then
      set K = B, n_x = n_x + n(C)
    else
      set K = C, k = k − n_ℓ(B)
    endif
  end case1
  begin case2: ρ(K) = a(B ⊡ x ⊡ C)
    if k ≤ 1 then
      set m = n(K) + n_x
      STOP
    elseif k ≤ 1 + n(B) then
      set K = B, k = k − 1
    else
      set K = C, k = k − 1 − n(B) − n_x
    endif
    set n_x = 0
  end case2
  begin case3: ρ(K) = B ⊡ C
    if k ≤ n(B) then
      set K = B
    else
      set K = C, k = k − n(B)
    endif
  end case3
```

```
  begin case4: ρ(K) = a(B)
    if k ≤ 1 then
      set m = n(K)
      STOP
    else
      set K = B, k = k − 1
    endif
  end case4
end loop
```

Design

This algorithm behaves like the algorithm for access and finds the node with pre-order number k_0 and sets $m = n(K)$. If $[\![K]\!]_F$ is a tree context then we also need to add the number n_x of nodes that would replace x.

4.6 Height

The height of a tree is the length of the longest path from the root to any leaf. The algorithm calculates the height m of the tree that is induced by the node with pre-order number $k_0 \in \mathbb{N}_1$ in the forest $[\![S]\!]_F$. If k_0 is not a pre-order number in the forest $[\![S]\!]_F$ then the algorithm terminates and sets $m = \varepsilon$.

Programming Variables

- k stores the local pre-order number
- K stores the current variable that needs to be evaluated
- n_x stores the number of nodes that would replace x in $\rho(K) = a(B \boxdot x \boxdot C)$
- m stores the height of the tree whose root is associated with k_0
- h_x stores the height of the tree that would replace x in $\rho(K) = a(B \boxdot x \boxdot C)$

Algorithm

```
set k = k0, K = S, nx = 0, m = ε, hx = 0
if k > n(K) then
  STOP
endif
begin loop:
  begin case1.1: t(K) = 1 and ρ(K) = B ◫ C
    if k ≤ nℓ(B) or k > nℓ(B) + n(C) + nx then
```

```
        set  h_x = max{h_s(C) + h_x, h(C)},  K = B,  n_x = n_x + n(C)
      else
        set  K = C,  k = k − n_ℓ(B)
      endif
    end  case1.1
    begin  case1.2:  t(K) = 0  and  ρ(K) = B ◫ C
      if  k ≤ n_ℓ(B)  or  k > n_ℓ(B) + n(C) + n_x  then
        set  h_x = h(C),  K = B,  n_x = n_x + n(C)
      else
        set  K = C,  k = k − n_ℓ(B)
      endif
    end  case1.2
    begin  case2:  ρ(K) = a(B ⊡ x ⊡ C)
      if  k ≤ 1  then
        set  m = max{1 + h_x, h(K)}
        STOP
      elseif  k ≤ 1 + n(B)  then
        set  K = B,  k = k − 1
      else
        set  K = C,  k = k − 1 − n(B) − n_x
      endif
      set  n_x = 0,  h_x = 0
    end  case2
    begin  case3:  ρ(K) = B ⊡ C
      if  k ≤ n(B)  then
        set  K = B
      else
        set  K = C,  k = k − n(B)
      endif
    end  case3
    begin  case4:  ρ(K) = a(B)
      if  k ≤ 1  then
        set  m = h(K)
        STOP
      else
        set  K = B,  k = k − 1
      endif
    end  case4
  end  loop
```

Design

This algorithm has a similar structure to the algorithm that computes access and finds the node with pre-order number k_0 and sets $m = h(K)$. If $[\![K]\!]_F$ is a tree context then we need a different approach. We introduce a programming variable h_x that stores the height of the tree that would replace x. It is practically used in the same manner as n_x.
If $\rho(K) = \underline{B} \boxplus C$ and $t(K) = 1$ then $[\![C]\!]_F$ is a tree context. We need to add the height of $[\![C]\!]_F$ to h_x. This cannot be done through a simple addition, because there are two possible longest paths in $[\![C]\!]_F$. If $h_x \neq 0$ then there is a forest with height h_x that would replace x in $[\![C]\!]_F$. Either, we follow along the spine of C and add the length of the spine $h_s(C)$ to h_x or there is a path in $[\![C]\!]_F$ with length $h(C)$ that diverges from the spine of C at some point. That explains why we need to consider the maximum of these two paths and set $h_x = max\{h_s(C) + h_x, h(C)\}$.
Case1.2 behaves exactly the same as case1.1, except that we set $h_x = h(C)$. The longest path has the length $h(C)$ and there is no longer path to consider. Also note that h_s is only defined for tree contexts.
Case2 uses the same argument as case1.1. If $\rho(K) = \underline{a}(B \boxdot x \boxdot C)$ then the height of $[\![K]\!]_F$ is the length of the longest path, so either $1 + h_x$ or $h(K)$.

4.7 Depth

The depth of a node is the length of the path from a unique root of the forest to that node. The algorithm calculates the depth m of the node with pre-order number $k_0 \in \mathbb{N}_1$ in the forest $[\![S]\!]_F$. If k_0 is not a pre-order number in the forest $[\![S]\!]_F$ then the algorithm terminates and sets $m = \varepsilon$.

Programming Variables

- k stores the local pre-order number
- K stores the current variable that needs to be evaluated
- n_x stores the number of nodes that would replace x in $\rho(K) = a(B \boxdot x \boxdot C)$
- m stores the depth of the node associated with k_0

Algorithm

set $k = k_0,\ K = S,\ n_x = 0,\ m = \varepsilon$
if $k > n(K)$ then
  STOP
else

```
  set m = 0
endif
begin loop:
  begin case1: ρ(K) = B ◫ C
    if k ≤ n_ℓ(B) or k > n_ℓ(B) + n(C) + n_x then
      set K = B, n_x = n_x + n(C)
    else
      set m = m + h_s(B), K = C, k = k − n_ℓ(B)
    endif
  end case1
  begin case2: ρ(K) = a(B ⊡ x ⊡ C)
    if k ≤ 1 then
      STOP
    elseif k ≤ 1 + n(B) then
      set K = B, k = k − 1
    else
      set K = C, k = k − 1 − n(B) − n_x
    endif
    set m = m + 1, n_x = 0
  end case2
  begin case3: ρ(K) = B ⊡ C
    if k ≤ n(B) then
      set K = B
    else
      set K = C, k = k − n(B)
    endif
  end case3
  begin case4: ρ(K) = a(B)
    if k ≤ 1 then
      STOP
    else
      set m = m + 1, K = B, k = k − 1
    endif
  end case4
end loop
```

Design

This algorithm behaves like the algorithm for access and finds the node with pre-order number k_0. Whenever we descend in the underlying forest $[\![S]\!]_F$ we need to

increment m. For example, if $\rho(K) = a(\underline{B})$ then we have to evaluate B next and increase the current depth m by 1.
If $\rho(K) = B \boxplus \underline{C}$ then we need to evaluate C next and increase the current depth m by $h_s(B)$.

4.8 Level Ancestor

The level ancestor of a node N_1 at a distance I is a node N_2 such that:

- the depth of N_2 is I
- there exists a path[7] from a unique root of the forest to N_1, that contains N_2

The algorithm calculates the pre-order number m of the ancestor of the node with pre-order number $k_0 \in \mathbb{N}_1$ at the distance $i_0 \in \mathbb{N}_0$ in the forest $[\![S]\!]_F$. If k_0 is not a pre-order number in the forest $[\![S]\!]_F$ or if $i_0 > \text{depth}(k_0)$ then the algorithm terminates and sets $m = \varepsilon$.

Programming Variables

- k stores the local pre-order number
- i stores the remaining distance
- K stores the current variable that needs to be evaluated
- n_x stores the number of nodes that would replace x in $\rho(K) = a(B \boxdot x \boxdot C)$
- m stores the pre-order number of the ancestor of the node associated with k_0
- M stores the variable B that contains the ancestor when $\rho(K) = B \boxplus C$

Algorithm

```
set  k = k_0, i = i_0, K = S, n_x = 0, m = ε, M = ε
if  k > n(K)  then
  STOP
endif
begin loop:
  begin case1: ρ(K) = B ⊞ C
    if  k ≤ n_ℓ(B)  or  k > n_ℓ(B) + n(C) + n_x  then
      set  K = B, n_x = n_x + n(C)
```

[7] In this definition we start at the root and descent I times, but there is an equivalent definition that starts at N_1 and ascends I' times. Both definitions are interchangeable. We can compute $I = \text{depth}(N_1) - I'$ and start the algorithm with $I \geq 0$.

```
    else
      if  $i < h_s(B)$  then
        set  $m = k_0 - k + 1,\ M = B$
        SEARCH
      else
        set  $K = C,\ k = k - n_\ell(B),\ i = i - h_s(B)$
      endif
    endif
  end case1
  begin case2: $\rho(K) = a(B \boxdot x \boxdot C)$
    if  $i \le 0$  then
      set  $m = k_0 - k + 1$
      STOP
    else
      if  $k \le 1$  then
        STOP
      elseif  $k \le 1 + n(B)$  then
        set  $K = B,\ k = k - 1$
      else
        set  $K = C,\ k = k - 1 - n(B) - n_x$
      endif
      set  $n_x = 0,\ i = i - 1$
    endif
  end case2
  begin case3: $\rho(K) = B \boxdot C$
    if  $k \le n(B)$  then
      set  $K = B$
    else
      set  $K = C,\ k = k - n(B)$
    endif
  end case3
  begin case4: $\rho(K) = a(B)$
    if  $i \le 0$  then
      set  $m = k_0 - k + 1$
      STOP
    else
      if  $k \le 1$  then
        STOP
      else
```

```
        set K = B, k = k − 1, i = i − 1
      endif
    endif
  end case4
end loop
beginfunction SEARCH:
  begin loop:
    begin case1: ρ(M) = B ◫ C
      if i < h_s(B) then
        set M = B
      else
        set i = i − h_s(B), m = m + n_ℓ(B), M = C
      endif
    end case1
    begin case2: ρ(M) = a(B ⊡ x ⊡ C)
      STOP
    end case2
  end loop
endfunction
```

Example

Let $F = (V, S, \rho)$ be the FSLP that we used in the example from chapter 2.4. We compute the pre-order number m of the level ancestor of the node with pre-order number $k_0 = 4$ at the distance $i_0 = 0$:

Initialization: $k = 4,\ i = 0,\ K = S,\ n_x = 0,\ m = \varepsilon,\ M = \varepsilon$

Iteration 1: case3: $\rho(K) = A \boxdot G$
$k = 4 \leq 6 = n(A)$
set $K = A$
Context: $k = 4,\ i = 0,\ K = A,\ n_x = 0,\ m = \varepsilon,\ M = \varepsilon$

Iteration 2: case1: $\rho(K) = A_1 \boxslash D$
$k = 4 > 3 = n_\ell(A_1)$ and $k = 4 \leq 4 = 3 + 1 + 0 = n_\ell(A_1) + n(D) + n_x$ and
$i = 0 < 2 = h_s(A_1)$
set $m = k_0 - k + 1 = 4 - 4 + 1 = 1,\ M = A_1$
SEARCH
Context: $k = 4,\ i = 0,\ K = A,\ n_x = 0,\ m = 1,\ M = A_1$

Iteration 3: case1: $\rho(M) = A_2 \boxplus B$
$i = 0 < 1 = h_s(A_2)$
set $M = A_2$
Context: $k = 4,\ i = 0,\ K = A,\ n_x = 0,\ m = 1,\ M = A_2$

Iteration 4: case2: $\rho(M) = a(Z \boxdot x \boxdot F)$
STOP

Design

This algorithm has a similar structure to the algorithm that computes access. We need to navigate through the FSLP and track the current depth in the underlying forest $[\![S]\!]_F$. We do this by decrementing i that is the remaining depth. Once we reach the depth i_0, which is equivalent to $i = 0$, the algorithm has found the level ancestor. Unfortunately, there are cases where we skip nodes entirely and cannot directly identify the level ancestor. For example, if $\rho(K) = B \boxplus \underline{C}$ and $i = 0$ then the node with pre-order number k_0 is produced by C, but the level ancestor is produced by B. The level ancestor is a node in the spine of B with the remaining depth $i = 0$. In this case, it is the root of $[\![B]\!]_F$. We need to set $M = B$ and $m = k_0 - k + 1$. M is used in the subroutine SEARCH that navigates through the spine of M and stops when the remaining depth is $i = 0$. The pre-order number of the first node produced by M is stored in m and is necessary to compute the pre-order number of the level ancestor.
Let us assume that $K = B$ and $\rho(B) = \underline{a}(B' \boxdot C')$. SEARCH calls STOP, because m is the pre-order number of a that is the first node with remaining depth $i = 0$ in the spine of B.
Let us consider the situation where we arrive at the node with pre-order number k_0 but the remaining depth is $i > 0$. In that case, there is no level ancestor and the algorithm calls STOP immediately. For example, if $\rho(K) = \underline{a}(B)$ and $i = 1$ then the level ancestor is found in B, which means the level ancestor is a descendant of a. That is a contradiction. The algorithm calls STOP and $m = \varepsilon$ by default, because we never updated m.

4.9 Nearest Common Ancestor

The algorithm calculates the pre-order number m of the deepest node that is the common ancestor of the nodes with pre-order numbers $k_{0,u}, k_{0,v} \in \mathbb{N}_1$ in the forest $[\![S]\!]_F$. If either $k_{0,u}$ or $k_{0,v}$ or both are not pre-order numbers in the forest $[\![S]\!]_F$ or if $k_{0,u}$ and $k_{0,v}$ are both roots then the algorithm terminates and sets $m = \varepsilon$. We can assume that $k_{0,u} \leq k_{0,v}$, otherwise swap both inputs.

Programming Variables

- k stores the local pre-order number during function SEARCH_SPINE
- k_0 stores either $k_{0,u}$ or $k_{0,v}$
  - if $k_{0,u}$ is constructed by B and $k_{0,v}$ is constructed by C when $\rho(K) = B \square\!\!\square C$ then $k_0 = k_{0,u}$
  - if $k_{0,v}$ is constructed by B and $k_{0,u}$ is constructed by C when $\rho(K) = B \square\!\!\square C$ then $k_0 = k_{0,v}$
  - the function SEARCH_SPINE finds the common ancestor in B
- k_u, k_v store the local pre-order numbers
- K stores the current variable that needs to be evaluated
- n_x stores the number of nodes that would replace x in $\rho(K) = a(B \boxdot x \boxdot C)$
- m stores the pre-order number of the common ancestor of the nodes associated with $k_{0,u}$ and $k_{0,v}$
- M stores the variable that contains the common ancestor
  - if $k_{0,u}$ is constructed by B and $k_{0,v}$ is constructed by C when $\rho(K) = B \boxdot C$ then the function SEARCH_PARENT finds the common ancestor in M
- m_1 stores the global pre-order number of the first node that is produced by M or stores the pre-order number of the common ancestor directly if $M = \varepsilon$

Algorithm

```
set k = ε, k_0 = ε, k_u = k_{0,u}, k_v = k_{0,v}
set K = S, n_x = 0, m = ε, M = ε, m_1 = ε
if k_u > n(K) or k_v > n(K) then
  STOP
endif
begin loop:
  begin case1: ρ(K) = B ◫ C
    if k_u ≤ n_ℓ(B) then
      if k_v ≤ n_ℓ(B) then
        set K = B, n_x = n_x + n(C)
      elseif k_v ≤ n_ℓ(B) + n(C) + n_x then
        set k = k_u, k_0 = k_{0,u}, K = B, n_x = n_x + n(C)
        SEARCH_SPINE
```

```
      else
        set  K = B, n_x = n_x + n(C)
      endif
    elseif  k_u ≤ n_ℓ(B) + n(C) + n_x  then
      if  n_ℓ(B) < k_v ≤ n_ℓ(B) + n(C) + n_x  then
        set  M = B, m_1 = k_{0,u} − k_u + 1, K = C
        set  k_u = k_u − n_ℓ(B), k_v = k_v − n_ℓ(B)
      else
        set  k = k_v, k_0 = k_{0,v}, K = B, n_x = n_x + n(C)
        SEARCH_SPINE
      endif
    else
        set  K = B, n_x = n_x + n(C)
    endif
  end case1
  begin case2: ρ(K) = a(B ⊡ x ⊡ C)
    if  k_u ≤ 1  then
      set  m = k_{0,u}
      STOP
    elseif  k_u ≤ 1 + n(B)  then
      if  k_v ≤ 1 + n(B)  then
        set  M = ε, m_1 = k_{0,u} − k_u + 1, K = B
        set  k_u = k_u − 1, k_v = k_v − 1
      else
        set  m = k_{0,u} − k_u + 1
        STOP
      endif
    else
      set  M = ε, m_1 = k_{0,u} − k_u + 1, K = C
      set  k_u = k_u − 1 − n(B) − n_x, k_v = k_v − 1 − n(B) − n_x
    endif
    set  n_x = 0
  end case2
  begin case3: ρ(K) = B ⊡ C
    if  k_u ≤ n(B)  then
      if  k_v ≤ n(B)  then
        set  K = B
      else
        SEARCH_PARENT
```

```
      endif
    else
      set  $K = C,\ k_u = k_u - n(B),\ k_v = k_v - n(B)$
    endif
  end case3
  begin case4: $\rho(K) = a(B)$
    if  $k_u \leq 1$  then
      set  $m = k_{0,u}$
      STOP
    else
      set  $M = \varepsilon,\ m_1 = k_{0,u} - k_u + 1,\ K = B$
      set  $k_u = k_u - 1,\ k_v = k_v - 1$
    endif
  end case4
end loop
beginfunction SEARCH_SPINE:
  begin loop:
    begin case1: $\rho(K) = B \boxplus C$
      if  $k \leq n_\ell(B)$  or  $k > n_\ell(B) + n(C) + n_x$  then
        set  $K = B,\ n_x = n_x + n(C)$
      else
        set  $K = C,\ k = k - n_\ell(B)$
      endif
    end case1
    begin case2: $\rho(K) = a(B \boxdot x \boxdot C)$
      set  $m = k_0 - k + 1$
      STOP
    end case2
  end loop
endfunction
beginfunction SEARCH_PARENT:
  if  $M \neq \varepsilon$  then
    begin loop:
      begin case1: $\rho(M) = B \boxplus C$
        set  $M = C,\ m_1 = m_1 + n_\ell(B)$
      end case1
      begin case2: $\rho(M) = a(B \boxdot x \boxdot C)$
        set  $m = m_1$
        STOP
```

```
      end case2
    end loop
  else
    set m = m_1
    STOP
  endif
endfunction
```

Example

Let $F = (V, S, \rho)$ be the FSLP that we used in the example from chapter 2.4. We compute the pre-order number m of the nearest common ancestor of the nodes with pre-order numbers $k_{0,u} = 3$ and $k_{0,v} = 6$:

Initialization:
$k = \varepsilon,\ k_0 = \varepsilon,\ k_u = 3,\ k_v = 6$
$K = S,\ n_x = 0,\ m = \varepsilon,\ M = \varepsilon,\ m_1 = \varepsilon$

Iteration 1: case3: $\rho(K) = A \boxdot G$
$k_u = 3 \leq 6 = n(A)$ and $k_v = 6 \leq 6 = n(A)$
set $K = A$
Context:
$k = \varepsilon,\ k_0 = \varepsilon,\ k_u = 3,\ k_v = 6$
$K = A,\ n_x = 0,\ m = \varepsilon,\ M = \varepsilon,\ m_1 = \varepsilon$

Iteration 2: case1: $\rho(K) = A_1 \boxplus D$
$k_u = 3 \leq 3 = n_\ell(A_1)$ and $k_v = 6 > 4 = 3 + 1 + 0 = n_\ell(A_1) + n(D) + n_x$
set $K = A_1,\ n_x = n_x + n(D) = 0 + 1 = 1$
Context:
$k = \varepsilon,\ k_0 = \varepsilon,\ k_u = 3,\ k_v = 6$
$K = A_1,\ n_x = 1,\ m = \varepsilon,\ M = \varepsilon,\ m_1 = \varepsilon$

Iteration 3: case1: $\rho(K) = A_2 \boxplus B$
$k_u = 3 > 1 = n_\ell(A_2)$ and $k_u = 3 \leq 5 = 1 + 3 + 1 = n_\ell(A_2) + n(B) + n_x$ and
$k_v = 6 > 5 = 1 + 3 + 1 = n_\ell(A_2) + n(B) + n_x$
set $k = k_v = 6,\ k_0 = k_{0,v} = 6,\ K = A_2,\ n_x = n_x + n(C) = 1 + 1 = 2$
SEARCH_SPINE
Context:
$k = 6,\ k_0 = 6,\ k_u = 3,\ k_v = 6$
$K = A_2,\ n_x = 2,\ m = \varepsilon,\ M = \varepsilon,\ m_1 = \varepsilon$

Iteration 4: SEARCH_SPINE case2: $\rho(K) = a(Z \boxdot x \boxdot F)$
set $m = k_0 - k + 1 = 6 - 6 + 1 = 1$
STOP

Design

This algorithm has a similar structure to the algorithm that computes access, but instead of tracking one local pre-order number we need to track two local pre-order numbers at the same time as we navigate through the FSLP. We are looking for the point where the pre-order numbers k_u and k_v are produced by two different terms with variables. At that point, it becomes a simple task to find the nearest common ancestor.

Let $\rho(K) = \underline{\underline{a}}(B \boxdot x \boxdot C)$. This means that $k_{0,u} = k_{0,v}$ and the pre-order number of the nearest common ancestor (NCA) is $m = k_{0,u}$. In fact, if a is the node with pre-order number $k_{0,u}$ then it does not matter which term with variables a, B or C produces the node with pre-order number $k_{0,v}$.

Let $\rho(K) = a(\underline{\underline{B}} \boxdot x \boxdot C)$. Since B produces the nodes with pre-order numbers $k_{0,u}$ and $k_{0,v}$ we need to set $K = B$ and $k_u = k_u - 1$ and $k_v = k_v - 1$ for further navigation. We also need to update M and m_1. Let us assume that $\rho(B) = \underline{B'} \boxdot \underline{C'}$ then it is possible that $k_{0,u}$ is produced by B' and $k_{0,v}$ is produced by C'. In that case the algorithm needs to call the subroutine SEARCH_PARENT. The pre-order number of the NCA a is $m_1 = k_{0,u} - k_u + 1$. We expect SEARCH_PARENT to set $m = m_1$ and call STOP immediately. That is why we need to set $M = \varepsilon$.

Let $\rho(K) = \underline{B} \boxplus \underline{C}$. Let us assume that $k_{0,u}$ is produced by B and $k_{0,v}$ is produced by C. The NCA must be a node produced by B, because it is an ancestor of x in $[\![B]\!]_F$. We set $k = k_u,\ k_0 = k_{0,u},\ K = B,\ n_x = n_x + n(C)$ and let SEARCH_SPINE find the ancestor. As the name implies we have to navigate through the spine of K and search for the node that is an ancestor of x and the ancestor of the node with pre-order number k_0. $[\![K]\!]_F$ is a tree context, so eventually SEARCH_SPINE will set $K = B'$ such that $\rho(B') = a(B'' \boxdot x \boxdot C'')$. We can set $m = k_0 - k + 1$ and call STOP, because a must be the NCA.

Let $\rho(K) = B \boxplus \underline{\underline{C}}$ and $\rho(C) = \underline{B'} \boxdot \underline{C'}$, where $k_{0,u}$ is produced by B' and $k_{0,v}$ is produced by C'. The NCA must be produced by B. SEARCH_PARENT needs to find the node that is the parent of x in $[\![B]\!]_F$. That is why we need to set $M = B$ and $m_1 = k_{0,u} - k_u + 1$ and navigate to $K = C$ and $k_u = k_u - n_\ell(B)$ and $k_v = k_v - n_\ell(B)$. The pre-order number of the first node that is produced by M is m_1.

4.10 Decompress

The algorithm calculates a term with variables M such that $[\![M]\!]_F$ is the tree that is induced by the node with pre-order number $k_0 \in \mathbb{N}_1$ in the forest $[\![S]\!]_F$. If k_0 is not a pre-order number in the forest $[\![S]\!]_F$ then the algorithm terminates and sets $M = \varepsilon$. A string replacement algorithm can then evaluate M using the definitions from chapter two.

Programming Variables

- k stores the local pre-order number
- K stores the current variable that needs to be evaluated
- n_x stores the number of nodes that would replace x in $\rho(K) = a(B \boxdot x \boxdot C)$
- M stores the term with variables that evaluates to the tree associated with k_0

Algorithm

```
set  k = k_0, K = S, n_x = 0, M = ε
if  k > n(K)  then
  STOP
endif
begin loop:
  begin case1: ρ(K) = B ◫ C
    if  k ≤ n_ℓ(B)  or  k > n_ℓ(B) + n(C) + n_x  then
      if  t(C) = 0  then
        set  M = C
      else
        set  M = C ◫ M
      endif
      set  K = B, n_x = n_x + n(C)
    else
      set  K = C, k = k − n_ℓ(B)
    endif
  end case1
  begin case2: ρ(K) = a(B ⊡ x ⊡ C)
    if  k ≤ 1  then
      set  M = K ◫ M
      STOP
    elseif  k ≤ 1 + n(B)  then
      set  K = B, k = k − 1
```

```
      else
        set  K = C, k = k − 1 − n(B) − n_x
      endif
      set  n_x = 0
    end  case2
    begin  case3: ρ(K) = B ⊡ C
      if  k ≤ n(B)  then
        set  K = B
      else
        set  K = C, k = k − n(B)
      endif
    end  case3
    begin  case4: ρ(K) = a(B)
      if  k ≤ 1  then
        set  M = K
        STOP
      else
        set  K = B, k = k − 1
      endif
    end  case4
end  loop
```

Design

This algorithm has a similar structure to the algorithm that computes access. As we navigate through the FSLP we need to compute the term with variables M that produces the tree with the root that is the node with pre-order number k_0. For example, if $\rho(K) = \underline{a}(B)$ then we set $M = K$ and call STOP.
If $\rho(K) = \underline{B} \boxplus C$ then we need to construct a term with variables of the form $M = Z \boxplus C \boxplus Z'$, where Z is a term with variables that produces a tree context and Z' is a term with variables that produces a forest.
If $t(C) = 0$ then $[\![C]\!]_F$ is a forest and we can set $M = C$ and continue with the construction in the next iteration.
If $t(C) = 1$ then $[\![C]\!]_F$ is a tree context and we can set $M = C \boxplus M$.
Let us assume that $K = B$, with $\rho(B) = \underline{a}(B' \boxdot x \boxdot C')$, then $[\![B]\!]_F$ is a tree context with a as the root and a is the node with pre-order number k_0. We can set $M = K \boxplus M$ and call STOP.

5 Conclusion

We have demonstrated that FSLPs support navigational queries. We have seen that we can compute a data structure in linear time such that the provided algorithms have a logarithmic runtime. It would be interesting to see an implementation in an actual programming language and an assessment of the practical use for tree based documents like XML. The provided algorithms could be useful to implement more complex queries like XPath. The proof of correctness is still an open problem and could be the topic of another thesis.

References

[1] M. Bojanczyk and I. Walukiewicz, "Forest algebras," in *Logic and Automata: History and Perspectives [in Honor of Wolfgang Thomas]* (J. Flum, E. Grädel, and T. Wilke, eds.), vol. 2 of *Texts in Logic and Games*, pp. 107–132, Amsterdam University Press, 2008.

[2] A. Gascón, M. Lohrey, S. Maneth, C. P. Reh, and K. Sieber, "Grammar-based compression of unranked trees," *Theory Comput. Syst.*, vol. 64, no. 1, pp. 141–176, 2020.

[3] P. Bille, I. L. Gørtz, G. M. Landau, and O. Weimann, "Tree compression with top trees," *Inf. Comput.*, vol. 243, pp. 166–177, 2015.

[4] M. Ganardi, A. Jez, and M. Lohrey, "Balancing straight-line programs," *J. ACM*, vol. 68, June 2021.

[5] M. Charikar, E. Lehman, D. Liu, R. Panigrahy, M. Prabhakaran, A. Sahai, and A. Shelat, "The smallest grammar problem," *IEEE Trans. Inf. Theory*, vol. 51, no. 7, pp. 2554–2576, 2005.

[6] C. G. Nevill-Manning and I. H. Witten, "Identifying hierarchical structure in sequences: A linear-time algorithm," *J. Artif. Intell. Res.*, vol. 7, pp. 67–82, 1997.

[7] T. A. Welch, "A technique for high-performance data compression," *Computer Magazine of the Computer Group News of the IEEE Computer Group Society*, vol. 17, no. 6, pp. 8–19, 1984.

[8] N. J. Larsson and A. Moffat, "Off-line dictionary-based compression," *Proceedings of the IEEE*, vol. 88, no. 11, pp. 1722–1732, 2000.

[9] M. Lohrey, "Algorithmics on slp-compressed strings: A survey," *Groups Complex. Cryptol.*, vol. 4, no. 2, pp. 241–299, 2012.

[10] M. Ganardi, "Compression by contracting straight-line programs," *CoRR*, vol. abs/2107.00446, 2021.

[11] C. P. Reh and K. Sieber, "Navigating forest straight-line programs in constant time," in *String Processing and Information Retrieval - 27th International Symposium, SPIRE 2020, Orlando, FL, USA, October 13-15, 2020, Proceedings* (C. Boucher and S. V. Thankachan, eds.), vol. 12303 of *Lecture Notes in Computer Science*, pp. 11–26, Springer, 2020.

Eidesstattliche Versicherung

Ich versichere nach § 39 (1) der Einheitlichen Regelung, meine Arbeit selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt sowie Zitate kenntlich gemacht zu haben.

Ort, Datum	Unterschrift